

DocAI 完整資料庫結構文檔 (修正版)

Complete Database Schema Documentation (Corrected)

生成日期: 2026-02-10 版本: v2.0 (已驗證) 系統版本: DocAI RAG Application v1.0.0 驗證狀態: ✓
已與源碼逐行比對

目錄 / Table of Contents

- [1. SQLite - Skill Metadata Database](#)
 - [2. SQLite - File Metadata Database](#)
 - [3. MongoDB - Chat History Database](#)
 - [4. Redis - Cache Layer](#)
 - [5. FAISS - Vector Store](#)
-

1. SQLite - Skill Metadata Database

檔案位置: /app/data/skill_metadata.db **Provider:** app/Providers/skill_metadata_provider/client.py **驅動:** aiosqlite (async SQLite) **源碼行號:** lines 140-320

1.1 skill_metadata 表 (技能文件元資料)

- 技能文件元資料表 - 存儲每個上傳文件的處理資訊
- Source: client.py lines 142-155

```
CREATE TABLE IF NOT EXISTS skill_metadata (  
    skill_id TEXT PRIMARY KEY,  
    skill_name TEXT NOT NULL,  
    skill_description TEXT,  
    skill_category TEXT,  
    skill_level TEXT DEFAULT 'intermediate',  
    tags TEXT, -- JSON 陣列格式  
    related_skills TEXT, -- JSON 陣列格式  
    total_chunks INTEGER DEFAULT 0,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    metadata TEXT -- JSON 格式的額外元資料  
);  
  
-- 後續透過 ALTER TABLE 添加的欄位  
-- Source: client.py lines 224-231  
ALTER TABLE skill_metadata ADD COLUMN head_id TEXT REFERENCES skill_heads(head_id);
```

```
-- 索引
-- Source: client.py lines 181-204
CREATE INDEX IF NOT EXISTS idx_skill_category ON skill_metadata(skill_category)
CREATE INDEX IF NOT EXISTS idx_skill_name ON skill_metadata(skill_name);
CREATE INDEX IF NOT EXISTS idx_skill_level ON skill_metadata(skill_level);
CREATE INDEX IF NOT EXISTS idx_skill_head_id ON skill_metadata(head_id);
```

1.2 skill_heads 表 (技能頭定義)

```
-- 技能分組頭表 - Single source of truth for skill definitions
-- Source: client.py lines 209-220

CREATE TABLE IF NOT EXISTS skill_heads (
    head_id TEXT PRIMARY KEY,
    skill_name TEXT NOT NULL UNIQUE,          -- 注意: UNIQUE 約束
    description TEXT,
    category TEXT DEFAULT 'General',
    display_order INTEGER DEFAULT 0,
    enabled BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

1.3 skill_overviews 表 (技能概述)

```
-- 技能概述摘要
-- Source: client.py lines 158-166

CREATE TABLE IF NOT EXISTS skill_overviews (
    skill_id TEXT PRIMARY KEY,
    overview TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id) ON DELETE CASCADE
);
```

1.4 skill_document_mapping 表 (文件映射)

```
-- 技能與文件的映射關係
-- Source: client.py lines 169-178

CREATE TABLE IF NOT EXISTS skill_document_mapping (
    mapping_id INTEGER PRIMARY KEY AUTOINCREMENT,
    skill_id TEXT NOT NULL,
    file_id TEXT NOT NULL,
```

```

    relevance_score REAL DEFAULT 0.0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id) ON DELETE CASCADE
);

```

```
-- 索引
```

```
-- Source: client.py lines 196-204
```

```

CREATE INDEX IF NOT EXISTS idx_skill_doc_mapping_skill ON skill_document_mapping
CREATE INDEX IF NOT EXISTS idx_skill_doc_mapping_file ON skill_document_mapping

```

1.5 skill_chunk_metadata 表 (Chunk 詳細元資料)

```
-- Chunk 層級的詳細元資料
```

```
-- Source: client.py lines 242-257
```

```

CREATE TABLE IF NOT EXISTS skill_chunk_metadata (
    chunk_id TEXT PRIMARY KEY,
    skill_id TEXT NOT NULL,
    document_id TEXT NOT NULL,
    document_name TEXT NOT NULL,
    page_number INTEGER,
    chunk_index INTEGER,
    chunk_text TEXT,
    embedding_model TEXT DEFAULT 'BAAI/bge-m3',
    embedding_dimension INTEGER DEFAULT 1024,
    metadata TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id) ON DELETE CASCADE
);

```

```
-- 索引
```

```
-- Source: client.py lines 259-268
```

```

CREATE INDEX IF NOT EXISTS idx_chunk_skill_id ON skill_chunk_metadata(skill_id)
CREATE INDEX IF NOT EXISTS idx_chunk_document_id ON skill_chunk_metadata(document_id)

```

1.6 processing_jobs 表 (處理任務)

```
-- 非同步處理任務追蹤 - 支援 checkpoint/resume 和容錯機制
```

```
-- Source: client.py lines 274-301
```

```

CREATE TABLE IF NOT EXISTS processing_jobs (
    job_id TEXT PRIMARY KEY,
    skill_id TEXT NOT NULL,
    head_id TEXT,
    pdf_path TEXT NOT NULL,
    pdf_filename TEXT,
    pdf_size_bytes INTEGER,
    total_pages INTEGER DEFAULT 0,

```

```

last_processed_page INTEGER DEFAULT 0,
total_chunks INTEGER DEFAULT 0,
processed_chunks INTEGER DEFAULT 0,
batch_size INTEGER DEFAULT 12,
status TEXT DEFAULT 'pending',           -- pending/processing/completed/fai
dirty_batches TEXT,                      -- JSON 陣列，記錄需要重試的批次
retry_count INTEGER DEFAULT 0,
error_message TEXT,
error_stack TEXT,
created_at TEXT DEFAULT CURRENT_TIMESTAMP,
updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
started_at TEXT,
completed_at TEXT,
processing_time_seconds REAL,
avg_page_time_ms REAL,
FOREIGN KEY (skill_id) REFERENCES skill_metadata(skill_id),
FOREIGN KEY (head_id) REFERENCES skill_heads(head_id)
);

-- 索引
-- Source: client.py lines 303-317
CREATE INDEX IF NOT EXISTS idx_pj_status ON processing_jobs(status);
CREATE INDEX IF NOT EXISTS idx_pj_skill ON processing_jobs(skill_id);
CREATE INDEX IF NOT EXISTS idx_pj_created ON processing_jobs(created_at);

```

2. SQLite - File Metadata Database

檔案位置: /app/data/docai.db **Provider:** app/Providers/

file_metadata_provider/client.py 驅動: aiosqlite (async SQLite) 源碼行號: lines 94-140

2.1 file_metadata 表 (檔案元資料)

```

-- File-Based RAG 系統的檔案元資料
-- Source: file_metadata_provider/client.py lines 96-108

CREATE TABLE IF NOT EXISTS file_metadata (
    file_id TEXT PRIMARY KEY,
    filename TEXT NOT NULL,           -- 注意: 非 file_name
    file_type TEXT NOT NULL,
    file_size INTEGER,
    upload_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP, -- 注意: 非 uploaded_at
    user_id TEXT,
    chunk_count INTEGER,             -- 注意: 非 total_chunks
    embedding_status TEXT,           -- 注意: 非 processing_status
    milvus_partition TEXT,
    metadata_json TEXT               -- 注意: 非 metadata
);

```

```
-- 索引
-- Source: file_metadata_provider/client.py lines 122-136
CREATE INDEX IF NOT EXISTS idx_file_user ON file_metadata(user_id);
CREATE INDEX IF NOT EXISTS idx_file_upload_time ON file_metadata(upload_time);
```

2.2 chunks_metadata 表 (Chunk 元資料)

```
-- File-Based 系統的 chunk 元資料
-- Source: file_metadata_provider/client.py lines 111-120

CREATE TABLE IF NOT EXISTS chunks_metadata (
    chunk_id TEXT PRIMARY KEY,
    file_id TEXT,
    chunk_index INTEGER,
    chunk_text TEXT, -- 注意: 非 content
    milvus_id INTEGER,
    FOREIGN KEY (file_id) REFERENCES file_metadata(file_id)
);

-- 索引
-- Source: file_metadata_provider/client.py lines 133-136
CREATE INDEX IF NOT EXISTS idx_chunk_file_id ON chunks_metadata(file_id);
```

3. MongoDB - Chat History Database

連線 URI: mongodb://mongodb:27017 (容器內) 或 mongodb://localhost:27017 (本機)
 資料庫名稱: docai 集合名稱: chat_sessions Provider: app/Providers/
 chat_history_provider/client.py 驅動: motor (async MongoDB) 源碼行號: lines
 70-145

3.1 chat_sessions 集合 (聊天會話)

```
// MongoDB 集合結構
// Source: chat_history_provider/client.py lines 126-134

// 文件結構:
{
    "_id": ObjectId("..."), // MongoDB 自動生成
    "session_id": "sess_20260210_uuid", // 會話唯一識別碼 (String)
    "user_id": "user_123", // 用戶識別碼 (String, Optional)
    "file_ids": ["file_1", "file_2"], // 注意: 陣列格式, 非單值
    "created_at": ISODate("..."), // 建立時間
    "updated_at": ISODate("..."), // 更新時間
    "messages": [ // 訊息陣列
        {
```

```

        "role": "user",                // 角色 (user/assistant/system)
        "content": "用戶問題...",
        "timestamp": ISODate("...")
    },
    {
        "role": "assistant",
        "content": "AI 回答...",
        "timestamp": ISODate("...")
    }
],
"metadata": {}                        // 額外元資料 (Object)
}

```

3.2 索引建立

```

// 索引定義
// Source: chat_history_provider/client.py lines 84-86

db.chat_sessions.createIndex({ "session_id": 1 }, { unique: true });
db.chat_sessions.createIndex({ "user_id": 1 });
db.chat_sessions.createIndex({ "created_at": 1 });

```

3.3 MongoDB 初始化腳本

```

// 初始化腳本 (可放入 docker-entrypoint-initdb.d/)
// File: init-mongo.js

use docai;

// 建立集合
db.createCollection("chat_sessions");

// 建立索引
db.chat_sessions.createIndex({ "session_id": 1 }, { unique: true });
db.chat_sessions.createIndex({ "user_id": 1 });
db.chat_sessions.createIndex({ "created_at": 1 });

print("MongoDB initialization completed for DocAI");

```

4. Redis - Cache Layer

連線 URI: redis://redis:6379/0 (容器內) 或 redis://localhost:6379/0 (本機)
Provider: app/Providers/cache_provider/client.py **驅動:** redis.asyncio (async Redis)
源碼行號: lines 1-493

4.1 快取策略 (Cache Strategy)

```
# Source: cache_provider/client.py lines 22-33

# Cache Strategy:
# - Embeddings: 24h TTL (expensive to compute)
# - Query Expansions: 1h TTL (moderate cost)
# - Search Results: 30min TTL (low cost but frequently accessed)
# - File Metadata: 6h TTL (semi-static data)

# Key Format:
# - emb:{text_hash} - Embedding vectors
# - qexp:{query_hash} - Query expansion results
# - search:{query_hash}:{file_ids} - Search results
# - file:{file_id} - File metadata
```

4.2 鍵值模式詳細說明

```
# =====
# 1. 嵌入向量快取 (Embedding Cache)
# =====
# Source: client.py lines 103-161
# 用途: 避免重複計算相同文字的嵌入向量
# TTL: settings.REDIS_EMBEDDING_TTL (預設 24 小時)

KEY_PATTERN = "emb:{text_hash}"
# text_hash = SHA256(text)[:16]

# 範例:
"emb:a1b2c3d4e5f6g7h8"

# 值格式: JSON 序列化的向量陣列
# [0.123, -0.456, 0.789, ...]

# =====
# 2. 查詢擴展快取 (Query Expansion Cache)
# =====
# Source: client.py lines 166-228
# 用途: 快取 LLM 產生的查詢擴展結果
# TTL: settings.REDIS_QUERY_EXPANSION_TTL

KEY_PATTERN = "qexp:{query_hash}"
# query_hash = SHA256(query)[:16]

# 值格式: JSON dict
# {"original_query": "...", "expanded_questions": [...], "intent": "..."}

# =====
# 3. 搜尋結果快取 (Search Results Cache)
# =====
# Source: client.py lines 233-318
```

```

# 用途：快取相同查詢的檢索結果
# TTL: settings.REDIS_SEARCH_RESULTS_TTL

KEY_PATTERN = "search:{cache_hash}"
# cache_hash = SHA256("{query}|{sorted_file_ids}|{top_k}")[:16]

# 值格式：JSON 陣列
# [{...}, {...}, ...]

# =====
# 4. 檔案元資料快取 (File Metadata Cache)
# =====
# Source: client.py lines 324-376
# 用途：快取檔案元資料
# TTL: 6 小時 (hardcoded 21600 秒)

KEY_PATTERN = "file:{file_id}"

# 值格式：JSON dict
# {"file_id": "...", "filename": "...", "chunk_count": 150, ...}

```

4.3 TTL 配置

```

# Source: client.py lines 60-63
# TTL 從 settings 配置讀取

self.embedding_ttl = settings.REDIS_EMBEDDING_TTL          # 24 小時
self.query_expansion_ttl = settings.REDIS_QUERY_EXPANSION_TTL # 1 小時
self.search_results_ttl = settings.REDIS_SEARCH_RESULTS_TTL # 30 分鐘

# File metadata TTL 硬編碼
# Source: client.py line 371
FILE_METADATA_TTL = 21600 # 6 小時

```

5. FAISS - Vector Store

儲存位置: /app/data/faiss_indices/ **Provider:** app/Providers/
vector_store_provider/client.py **驅動:** faiss-cpu +
langchain_community.vectorstores **源碼行號:** lines 240-340

5.1 FAISS 索引結構

```

# Source: vector_store_provider/client.py lines 255-283

import faiss
from langchain_community.vectorstores import FAISS

```



```

from langchain_community.docstore.in_memory import InMemoryDocstore

# =====
# 索引類型: IndexFlatL2 (L2 距離精確搜尋)
# =====

def create_faiss_index(precomputed_embeddings):
    """建立 FAISS 向量索引"""

    # 維度從嵌入向量動態獲取
    dimension = precomputed_embeddings.shape[1]

    # IndexFlatL2: 精確 L2 距離搜尋 (無壓縮)
    index = faiss.IndexFlatL2(dimension)

    # 批次添加優化 (大型資料集)
    faiss_batch_size = 10000
    total_vectors = len(precomputed_embeddings)

    if total_vectors <= faiss_batch_size:
        # 小型資料集: 一次添加
        index.add(precomputed_embeddings)
    else:
        # 大型資料集: 分批添加以減少記憶體壓力
        for i in range(0, total_vectors, faiss_batch_size):
            batch = precomputed_embeddings[i:i + faiss_batch_size]
            index.add(batch)

    return index

```

5.2 BGE Embedding Wrapper

Source: vector_store_provider/client.py lines 300-318

LangChain-compatible wrapper for BGE embeddings

```

class BGEWrapper(LCEmbeddings):
    def __init__(self, provider):
        self._provider = provider

    def embed_query(self, text: str):
        return self._provider.embed_single(text).tolist()

    def embed_documents(self, texts):
        return [self._provider.embed_single(t).tolist() for t in texts]

```

5.3 目錄結構

```

/app/data/faiss_indices/
├── skills/                                # Skill-Based 索引 (BGE-M3, 1024 維)
│   ├── skill_20260210_abc123/
│   │   └── index.faiss                    # FAISS 二進位索引檔

```

```

├── index.pkl          # LangChain docstore (pickle)
├── skill_20260210_def456/
│   ├── index.faiss
│   └── index.pkl
└── ...

└── files/            # File-Based 索引 (MiniLM, 384 維)
    ├── file_20260210_xyz789/
    │   ├── index.faiss
    │   └── index.pkl
    └── ...

```

5.4 store_type 隔離

Source: vector_store_provider/client.py lines 338-339

store_type 參數用於物理隔離

"skill" -> /app/data/faiss_indices/skills/{skill_id}/

"file" -> /app/data/faiss_indices/files/{file_id}/

self._save_faiss_store(store_id, vector_store, store_type=store_type)

5.5 索引檔案格式

index.faiss (二進位格式)

```

├── 索引類型標識 (IndexFlatL2)
├── 向量維度 (dimension)
├── 向量數量 (ntotal)
└── 向量資料 (float32 陣列)

```

index.pkl (Pickle 序列化)

```

├── docstore: InMemoryDocstore
│   └── {id: Document(page_content, metadata)}
└── index_to_docstore_id: {faiss_idx: doc_id}

```

6. 環境變數參考

```

# =====
# SQLite 設定
# =====
SQLITE_DB_PATH=/app/data/skill_metadata.db
DATABASE_URL=sqlite:///app/data/skill_metadata.db

# =====
# MongoDB 設定
# =====
MONGODB_URI=mongodb://mongodb:27017          # 容器內

```

```

MONGODB_DATABASE=docai
MONGODB_COLLECTION=chat_history

# =====
# Redis 設定
# =====
REDIS_URL=redis://redis:6379/0          # 容器內
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_DB=0

# Redis TTL 設定 (從 settings 讀取)
REDIS_EMBEDDING_TTL=86400              # 24 小時
REDIS_QUERY_EXPANSION_TTL=3600         # 1 小時
REDIS_SEARCH_RESULTS_TTL=1800         # 30 分鐘

# =====
# FAISS / Vector Store 設定
# =====
VECTOR_STORE_TYPE=faiss
VECTOR_STORE_PATH=/app/data/faiss_indices

# Skill-Based: BGE-M3
EMBEDDING_MODEL_BGE_M3=BAAI/bge-m3
EMBEDDING_DIMENSION_BGE_M3=1024

# File-Based: MiniLM
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
EMBEDDING_DIMENSION=384

```

7. 初始化順序

7.1 容器啟動流程

```

# docker-compose.yml 依賴關係

services:
  docai-app:
    depends_on:
      mongodb:
        condition: service_healthy    # 1. 等待 MongoDB 健康
      redis:
        condition: service_healthy    # 2. 等待 Redis 健康
    # 3. DocAI 應用啟動時自動初始化 SQLite 和 FAISS 目錄

```

7.2 應用程式初始化

```
# main.py 初始化邏輯

async def initialize_databases():
    # 1. SQLite - 自動建立表格
    skill_provider = SkillMetadataProvider()
    await skill_provider.initialize() # 建立 skill_metadata.db 所有表

    file_provider = FileMetadataProvider()
    await file_provider.initialize() # 建立 docai.db 所有表

    # 2. MongoDB - 建立連線和索引
    chat_provider = ChatHistoryProvider()
    await chat_provider._get_collection() # 自動建立索引

    # 3. Redis - 建立連線 (lazy initialization)
    cache_provider = CacheProvider()
    await cache_provider._get_redis()

    # 4. FAISS - 確保目錄存在
    os.makedirs("/app/data/faiss_indices/skills", exist_ok=True)
    os.makedirs("/app/data/faiss_indices/files", exist_ok=True)
```

驗證狀態

組件	驗證狀態	源碼行號
skill_metadata	✓ 已驗證	lines 142-155
skill_heads	✓ 已驗證	lines 209-220
skill_overviews	✓ 已驗證	lines 158-166
skill_document_mapping	✓ 已驗證	lines 169-178
skill_chunk_metadata	✓ 已驗證	lines 242-257
processing_jobs	✓ 已驗證	lines 274-301
file_metadata	✓ 已驗證	lines 96-108
chunks_metadata	✓ 已驗證	lines 111-120
MongoDB chat_sessions	✓ 已驗證	lines 126-134, 84-86
Redis cache	✓ 已驗證	lines 22-33, 60-63
FAISS	✓ 已驗證	lines 255-283

文檔結束

*Generated by SuperClaude Framework for DocAI Containerization Project Version: v2.0
(Corrected and Validated) Validation Date: 2026-02-10*